

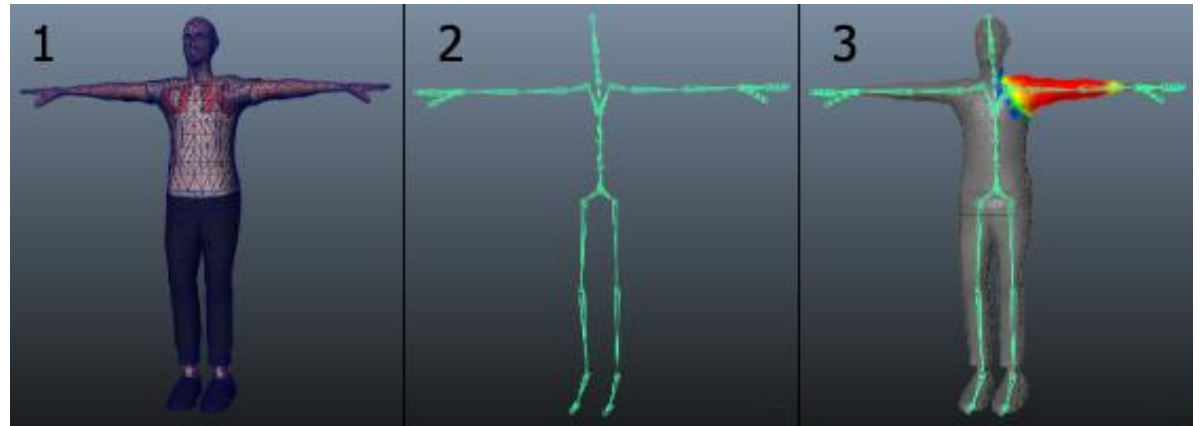
Video Game Programming: Characters and Animation

L10

Prof. Fabio Buttussi, PhD
Human-Computer Interaction Laboratory
Department of Mathematics, Computer Science, and Physics
University of Udine - Italy

Game characters

- To use characters with animations in 3D games the following steps are needed (in fig.):
 1. Modeling
 2. Rigging
 3. Skinning



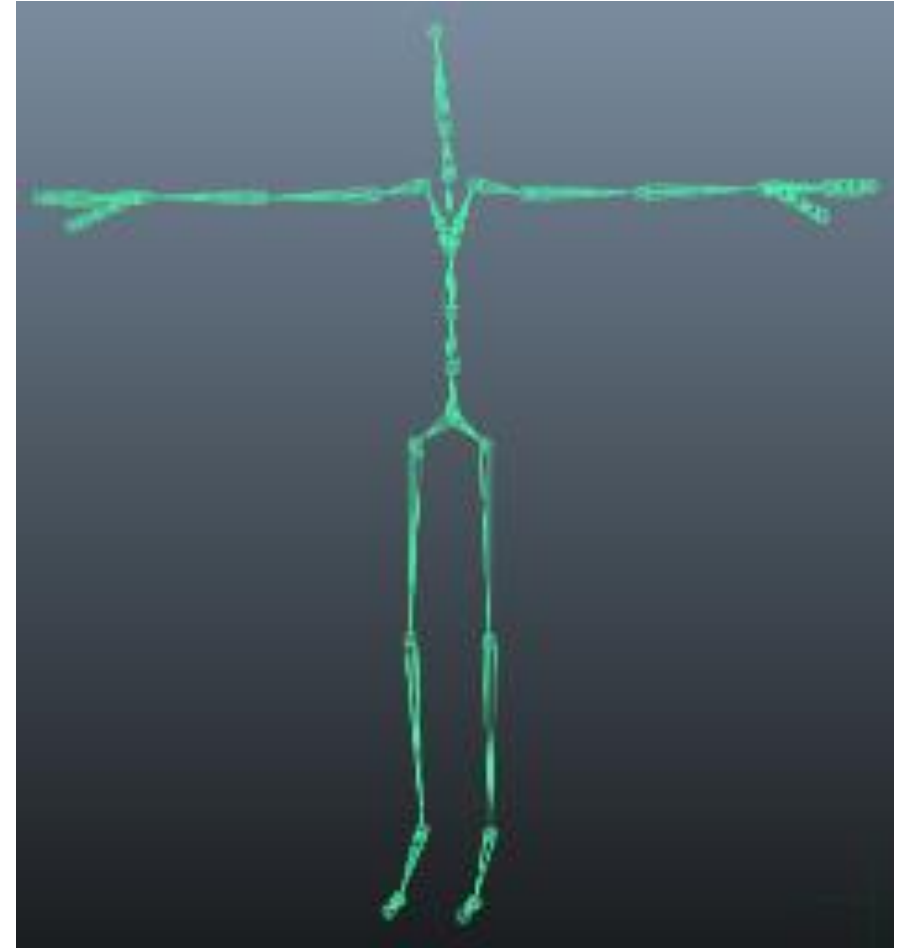
Character modeling

- It consists in creating a humanoid mesh with 3D modeling software (e.g., 3ds Max, Blender) or character generators (e.g., Makehuman, Character Creator, MetaHuman)
- Characters can also be acquired on 3D models and game assets stores (e.g., cgtrader, Unity Asset Store)
- Microsoft released Rocketbox set of characters under MIT license
- A character is generally made of polygons and in T-pose (in fig.)



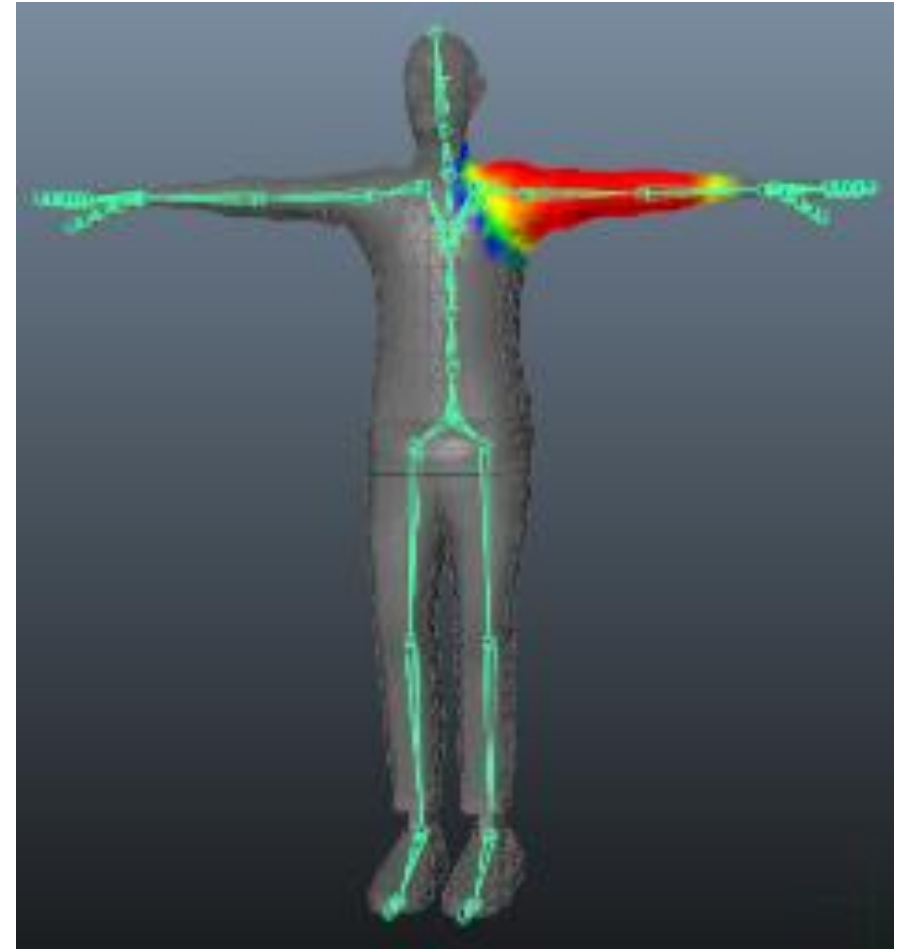
Rigging

- It consists in creating a skeleton to control character movements
- The skeleton is a hierarchy of joints to define the bones inside the mesh and their relative movement (in fig.)
- Some software and online services offer standard biped skeletons (e.g., 3ds Max) or semi-automatic rigging (e.g., Mixamo)
- Fake bones can be added to animate face or props



Skinning

- It consists in attaching mesh to skeleton by defining which polygons move when a joint is animated
- Each vertex can be matched to one or more bones with weights usually represented as heatmaps (in fig.)
- Some online services and software (e.g., Mixamo, Maya) allow to semi-automatically create or manually fine-tune skinning



Character animation

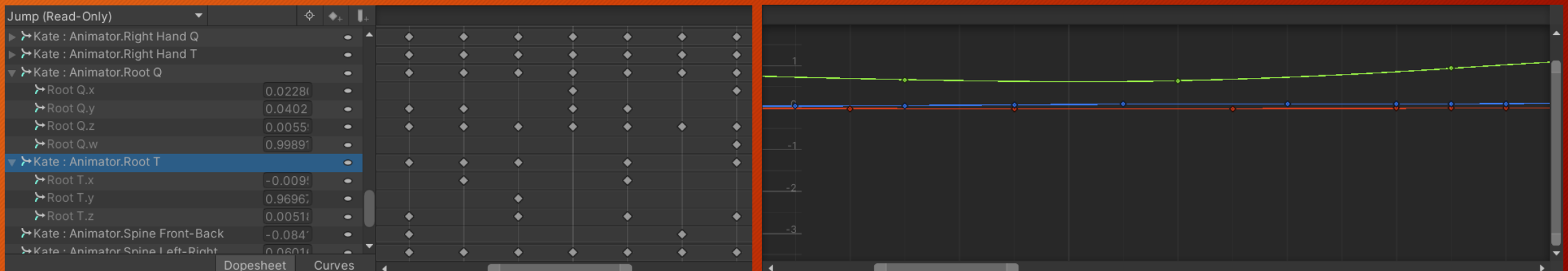
- Animation of characters can be done/acquired in 3D modeling/animation software/services or implemented in the game engine
- Different techniques can be used:
 - Forward kinematics
 - Inverse kinematics
 - Blend shapes
 - Physics



Frame of a Mixamo animation

Forward kinematics (FK)

- It consists in specifying rotation (and position) of joints in important instants of time called **keyframes**
- Joint values between two keyframes are determined by interpolation using curves
- Values can be inserted manually or using motion capture



Inverse kinematics (IK)

- It consists in specifying the desired positions of end effectors (e.g., hands, feet)
- Joint rotations to match those positions are calculated (solved) automatically
- Animation software can plot IK takes to FK keyframes
- Some game engines support IK solving in real-time using scripts



IK in MotionBuilder

Blend shapes

- Also known as morph target or per-vertex animation
- It consists in setting the weights for different shapes of a mesh for each keyframe
- Using weights, it interpolates between the shapes at each keyframe and between keyframes
- It is typically used for facial animation (in fig.), but can be used for other parts of characters or generic objects



Lewis et al. (2014), Practice and Theory of Blendshape Facial Models. *Eurographics (State of the Art Reports)*, 1(8), 2.

Physics

- A character can also be animated using physics
- Physics is typically used to simulate falls (ragdoll) as in fig.
- Game engines such as Unity have specific wizards to create ragdolls for characters
- More complex physics-based animation is available with advanced tools such as ZivaRT



Recap reference

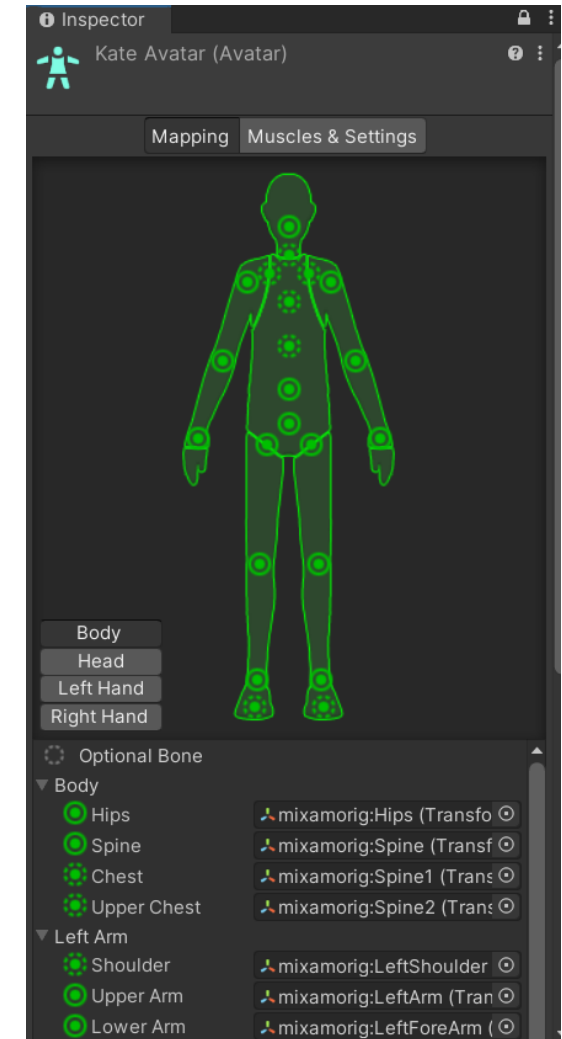
- Gonzalez-Franco et al., 2020, The Rocketbox Library and the Utility of Freely Available Rigged Avatars, Front. Virtual Real. 1, 20, <https://doi.org/10.3389/frvir.2020.561558>
- Besides introducing Rocketbox library of free avatars, it includes a recap about:
 - Character creation
 - Character texturing
 - Different types of character animation
 - Comparison of character resources
 - Use of characters in different studies and applications

Animation in games

- Game engines offer library methods and tools to:
 - play (and create) FK animations
 - use IK to change position of end effectors
 - set weights of blend shapes
 - handle ragdoll physics
- Typically, games use:
 - IK for interaction with objects
 - blend shapes for facial expression and speech
 - ragdoll physics for falls
 - FK for all the other animations

Animation retargeting

- Animations for a character cannot be used on other ones with different joint names
- If joint names are the same, but positions differ, the animation will look weird
- Some game engine use intermediate representations to retarget animations
- Unity has an intermediate muscle-based character representation called **Avatar**, used to retarget animations and enable IK

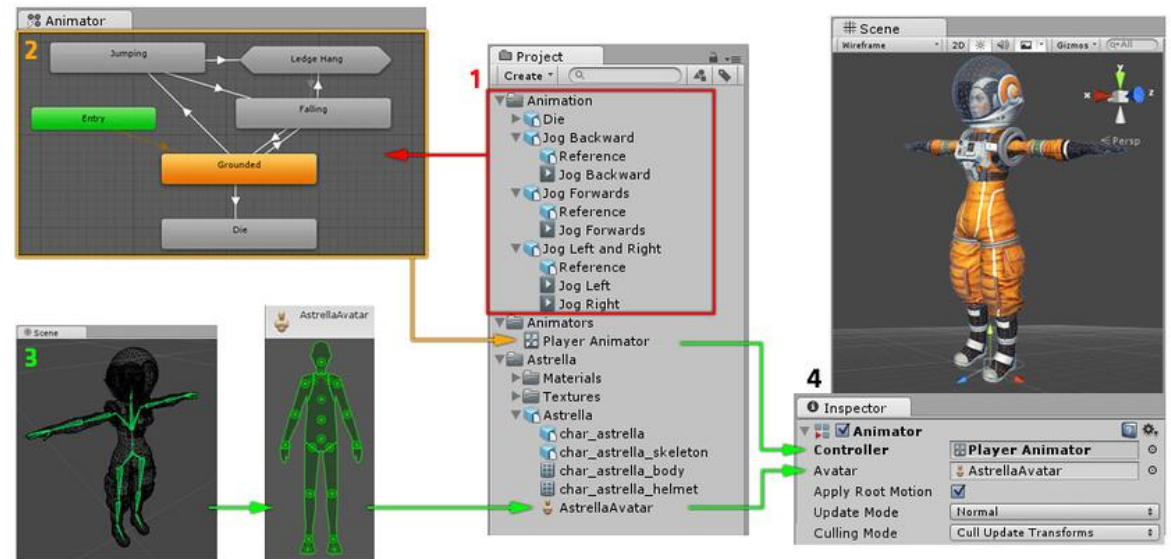


Animation transition

- A specific FK or blend shape animation (sequence of keyframes with timing) can be called **animation clip**
- Playing separate clips is not optimal in a game because
 - characters should not instantaneously change joint or blend shapes values between a clip and another
 - it is almost impossible to make all possible animations between clips as additional clips
- Game engines thus offer functions to fade between a clip and another and/or tools to handle animation transitions

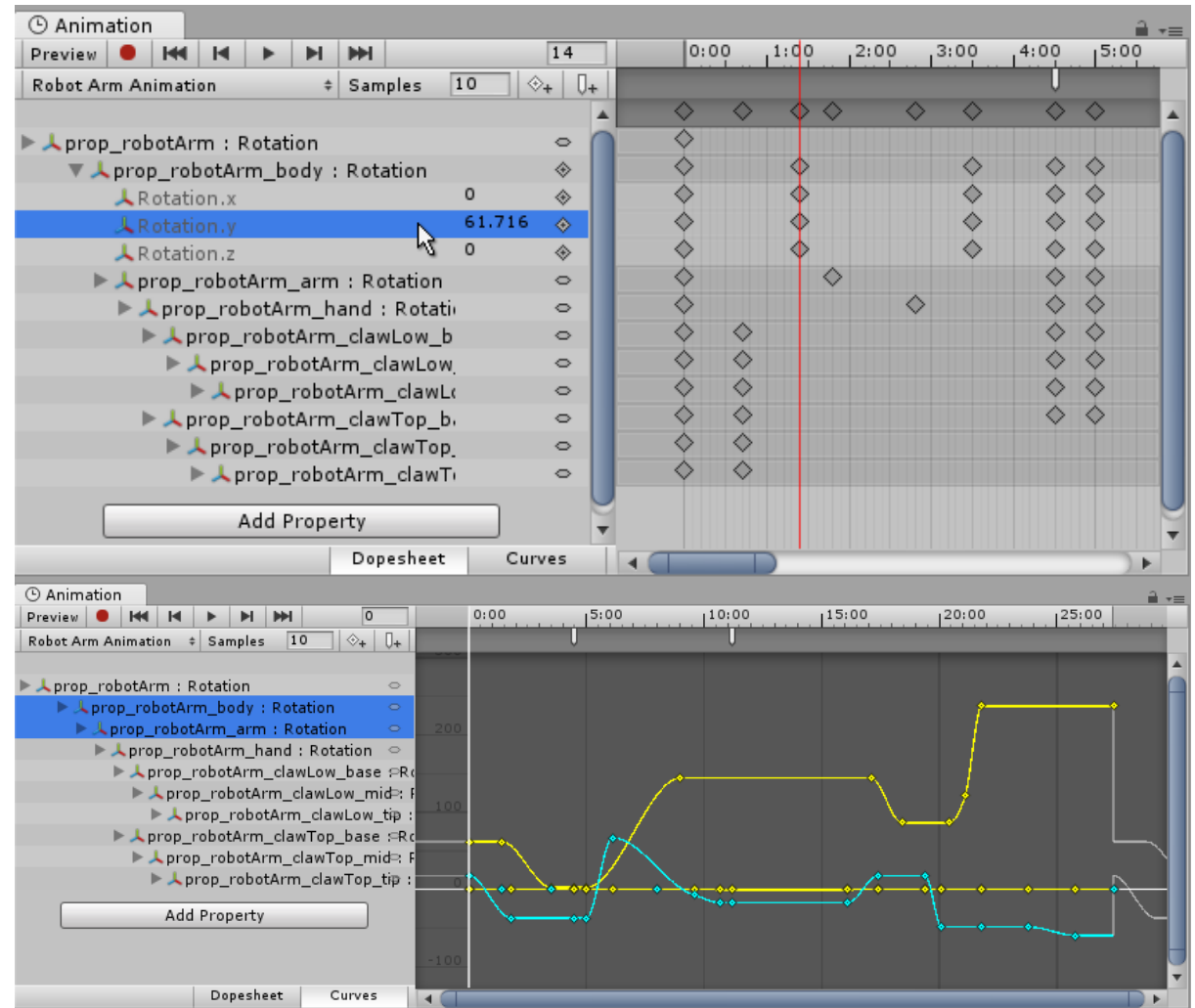
Mecanim

- It is the articulated Unity animation system
- In addition to Avatar, it includes:
 - An Animation window to edit clips
 - An Animator window to edit Animation Controllers that organize animations and their transitions



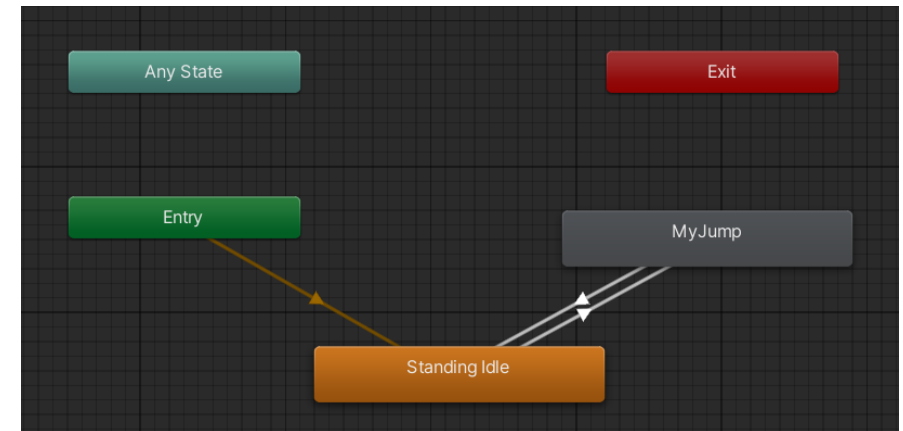
Animation window

- It allows to:
 - add joints to animate
 - set joint values
 - change keyframe timing
 - edit animation curves
 - record animations
 - add events
- Events allow to call functions at specific times of the animation



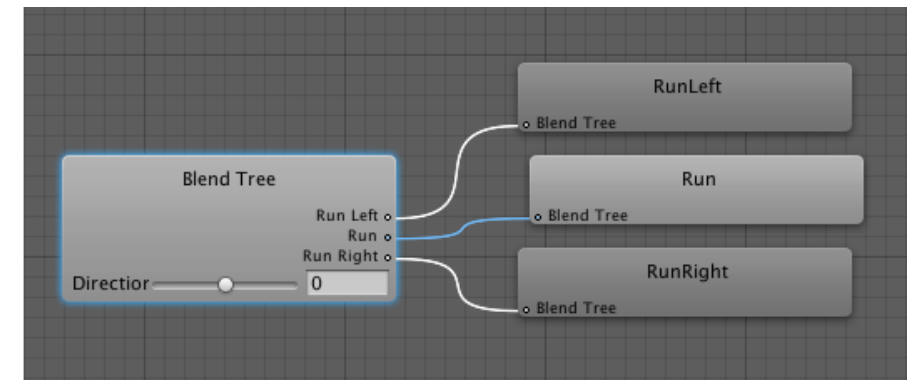
Animator states

- Animator window organizes animation clips as states of a DFA or **state machine**
- Animation transitions are mapped to state transitions
- Special states without animation define entry/exit states and an “any state” represents them all
- States can have substates in nested state machines or blend animations in blend trees



Blend trees

- A blend tree allows to blend two or more animations into a new animation
- The weight of each animation on the final animation can be controlled using a parameter
- They are particularly useful to blend similar animation (e.g., walk to run, walk in different directions)



Animator parameters and layers

- Animator parameters allow to specify values that can be combined in conditions to determine transitions from a state to another
- In addition to float, int, and bool, parameters could be of type trigger, automatically deactivated after a transition using it
- An animation controller can be organized in multiple independent layers (distinct state machines), e.g., to upper and low body parts
- Each layer is associated to a mask that indicates which parts of the avatar are controlled by that layer
- A special flag determines if IK can pass through the layer

Scripting animations

- Animation can be controlled using scripts by:
 - Changing parameters of animator
 - Cross fading to a specific state
 - Setting weights of blend shapes
 - Setting weights and targets of IK
- Functions can be activated by events
- States in state machines can have State Machine Behaviours to execute code when entering/leaving a state

Animation of other objects

- Unity uses Animation and Animator windows also to animate other game objects than characters
- Animation window can edit component properties of different types (i.e., Float, Color, Vector2/3/4, Quaternion, Boolean)
- Animator component (without an avatar) can use a controller to handle transitions between these animations
- This is particularly useful to animate the UI

Bibliografy

- Gomila Salas Juan Gabriel, Borromeo Nicolas Alejandro, Hands-On Unity Game Development, 4th ed., Packt Publishing, 2024
- Murray, C# Game Programming Cookbook for Unity 3D, 2nd ed., CRC Press, 2021

